

# Exp 6 a: First Come First Served

**FACE Prep**

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

**First-Come,First-Served CPU Scheduling Algorithm**

Completed

Write a C program to implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

- **Completion Time (CT):** The time at which a process finishes execution.
- **Turnaround Time (TAT):** Total time taken from arrival to completion.
- $TAT = CT - AT$
- **Waiting Time (WT):** The time a process spends waiting in the ready queue.
- $WT = TAT - BT$

Select Language: C (GCC 9.2.0)

```
1 #include <stdio.h>
2 int main()
3 {
4     int n, i;
5     scanf("%d",&n);
6     int AT[n],BT[n],CT[n],TAT[n],WT[n];
7     float avgTAT=0,avgWT=0;
8     printf("Enter number of processes: ");
9     for(i=0;i<n;i++){
10         printf("\nEnter Arrival Time and Burst Time for P%d: ",i+1);
11         scanf("%d%d",&AT[i],&BT[i]);
12     }
13     CT[0] = AT[0] + BT[0];
14     for(i=1;i<n;i++){
15         if(CT[i-1] < AT[i])
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

**FACE Prep**

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

**Output Format**

The output displays the Average Turnaround Time and the Average Waiting Time.

**Example 1**

**Sample Input 1**

3  
0 2  
1 2  
2 3

**Sample Output 1**

Enter number of processes:  
Enter Arrival Time and Burst Time for P1:  
Enter Arrival Time and Burst Time for P2:  
Enter Arrival Time and Burst Time for P3:

Select Language: C (GCC 9.2.0)

```
15 {
16     if(CT[i-1] < AT[i])
17         CT[i] = AT[i] + BT[i];
18     else
19         CT[i] = CT[i-1] + BT[i];
20 }
21 for(i=0;i<n;i++){
22 {
23     TAT[i] = CT[i] - AT[i];
24     WT[i] = TAT[i] - BT[i];
25     avgTAT += TAT[i];
26     avgWT += WT[i];
27 }
28 avgTAT /= n;
29 avgWT /= n;
30 printf("\n\nAverage Turnaround Time: %.2f",avgTAT);
31 printf("\n\nAverage Waiting Time: %.2f",avgWT);
32 return 0;
33 }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Input:

Expected Output:

# Exp 6 b: Non – Preemptive Shortest Job First

**FACE Prep**

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Save

**Non-Preemptive Shortest Job First CPU Scheduling Algorithm**

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

**Core Logic**

**Selection:** Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

**Formulae:**

- $\text{Turnaround Time (TAT)} = \text{Completion Time (CT)} - \text{Arrival Time (AT)}$
- $\text{Waiting Time (WT)} = \text{Turnaround Time (TAT)} - \text{Burst Time (BT)}$

**Input Format**

1. The first line of input contains an integer representing the number of processes.

Completed

Select Language: C (GCC 9.2.0)

```
1 #include<stdio.h>
2 int main(){
3     int n,i,j,time=0,min,completed=0;
4     int at[20],bt[20],ct[20],tat[20],wt[20];
5     int visited[20]={0};
6     float avg_tat=0,avg_wt=0;
7     printf("Enter number of processes: ");
8     scanf("%d",&n);
9     for(i=0;i<n;i++){
10        printf("\nEnter Arrival Time and Burst Time for P%d: ",i+1);
11        scanf("%d %d",&at[i],&bt[i]);
12    }
13    while(completed<n){
14        min=-1;
15        for(i=0;i<n;i++){
16            if(at[i]<=time && visited[i]==0){
17                if(min== -1 || bt[i]<bt[min]){
18                    min=i;
19                }
20            }
21        }
22        if(min != -1){
23            time+=bt[min];
24            ct[min]=time;
25            tat[min]=ct[min]-at[min];
26            wt[min]=tat[min]-bt[min];
27            avg_tat+=tat[min];
28            avg_wt+=wt[min];
29            visited[min]=1;
30            completed++;
31        }
32    }
33    avg_tat/=n;
34    avg_wt/=n;
35    printf("\n\nAverage Turnaround Time = %.2f\n",avg_tat);
36    printf("Average Waiting Time = %.2f\n",avg_wt);
37    return 0;
38 }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

**FACE Prep**

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Save

**Output Format**

The output displays the Average Turnaround Time and the Average Waiting Time.

**Example 1**

**Sample Input 1**

4

0 6

0 8

0 7

0 3

**Sample Output 1**

Enter number of processes:

Enter Arrival Time and Burst Time for P1:

Enter Arrival Time and Burst Time for P2:

Enter Arrival Time and Burst Time for P3:

Select Language: C (GCC 9.2.0)

```
23 time++;
24 }
25 else{
26     time +=bt[min];
27     ct[min]=time;
28     tat[min]=ct[min]-at[min];
29     wt[min]=tat[min]-bt[min];
30     avg_tat+=tat[min];
31     avg_wt+=wt[min];
32     visited[min]=1;
33     completed++;
34 }
35 }
36 avg_tat/=n;
37 avg_wt/=n;
38 printf("\n\nAverage Turnaround Time = %.2f\n",avg_tat);
39 printf("Average Waiting Time = %.2f\n",avg_wt);
40 return 0;
41 }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

# Exp 6 c: Non – Preemptive Priority Scheduling

**FACE Prep**

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Save

Completed

**Non-Preemptive Priority Scheduling algorithm**

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

**Input Format**

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

**Output Format**

The output displays the Average Turnaround Time and the Average Waiting Time.

Select Language : C (GCC 9.2.0)

```
1 #include<stdio.h>
2 int main()
3 {
4     int n;
5     printf("Enter number of processes: \n");
6     scanf("%d",&n);
7     int at[n],bt[n],pr[n];
8     int ct[n],tat[n],wt[n];
9     int done[n];
10    for(int i=0;i<n;i++){
11        done[i]=0;
12    }
13    for(int i=0;i<n;i++){
14        {
15            printf("Enter AT, BT and Priority for P%d: \n",i+1);
16            scanf("%d %d %d",&at[i],&bt[i],&pr[i]);
17        }
18        int time=0,completed=0;
19        while(completed<n)
20        {
21            int idx=-1;
22            for(int i=0;i<n;i++){
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

**FACE Prep**

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Save

Example 1

**Sample Input 1**

3

0 5 2

1 3 1

2 2 3

**Sample Output 1**

Enter number of processes:

Enter AT, BT and Priority for P1:

Enter AT, BT and Priority for P2:

Enter AT, BT and Priority for P3:

Average Turnaround Time = 6.67

Average Waiting Time = 3.33

Select Language : C (GCC 9.2.0)

```
32     time+=bt[idx];
33     ct[idx]=time;
34     tat[idx]=ct[idx]-at[idx];
35     wt[idx]=tat[idx]-bt[idx];
36     done[idx]=1;
37     completed++;
38 }
39 else
40     time++;
41 }
42 float avg_tat=0,avg_wt=0;
43 for(int i=0;i<n;i++){
44     avg_tat+=tat[i];
45     avg_wt+=wt[i];
46 }
47 avg_tat/=n;
48 avg_wt/=n;
49 printf("\n");
50 printf("Average Turnaround Time = %.2f\n",avg_tat);
51 printf("Average Waiting Time = %.2f\n",avg_wt);
52 return 0;
53 }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

# Exp 6 d: Round Robin

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

Round Robin CPU Scheduling Algorithm

Completed

Write a C program to implement the Round Robin (RR) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), Burst Times (BT), and a Time Quantum (TQ). The CPU should rotate through the processes in the ready queue, granting each a maximum of one TQ of execution time. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and the overall averages.

Input Format

1. The first line of input contains an integer representing the number of processes.  
2. The second line contains an integer representing the Time Quantum.  
3. The next lines contain two integers for each process, where  

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Select Language: C (GCC 8.2.0)

```
1 #include<stdio.h>
2 int main(){
3     int n,tq;
4     int at[20], bt[20], rt[20];
5     int ct[20], tat[20], wt[20];
6     int i,time=0, completed=0;
7     float avg_tat=0,avg_wt=0;
8     printf("Enter number of processes: \n");
9     scanf("%d", &n);
10    printf("Enter Time Quantum: \n");
11    scanf("%d",&tq);
12    for(i=0;i<n;i++){
13        printf("Enter AT and BT for P%d: \n",i+1);
14        scanf("%d %d",&at[i],&bt[i]);
15        rt[i]=bt[i];
16    }
17    while(completed<n){
18        int executed = 0;
19        for(i=0;i<n;i++){
20            if(rt[i]>0 && at[i]<=time){
21                executed=i;
22                if(rt[i]>tq){
```

Test Cases 2

Run Sample Cases

Run All Cases

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Sample Input 1

3  
2  
0 5  
1 3  
2 1

Sample Output 1

Enter number of processes:  
Enter Time Quantum:  
Enter AT and BT for P1:  
Enter AT and BT for P2:  
Enter AT and BT for P3:  
  
Average Turnaround Time: 6.33  
Average Waiting Time: 3.33

Select Language: C (GCC 8.2.0)

```
29         rt[i]=0;
30         completed++;
31     }
32 }
33 }
34 if(executed==0){
35     time++;
36 }
37 }
38 for(i=0;i<n;i++){
39     tat[i]=ct[i]-at[i];
40     wt[i]=tat[i]-bt[i];
41     avg_tat+=tat[i];
42     avg_wt+=wt[i];
43 }
44 avg_tat=avg_tat/n;
45 avg_wt=avg_wt/n;
46 printf("\n");
47 printf("Average Turnaround Time: %.2f\n",avg_tat);
48 printf("Average Waiting Time: %.2f\n",avg_wt);
49 return 0;
50 }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1